Gestion des Processus et Signaux POSIX sous Unix/Linux

Thème principal : Systèmes d'exploitation et fonctionnement (Processus, Ordonnancement, IPC)

Synthèse exécutive : Ce cours formalise la transition critique entre un programme statique stocké sur disque et un processus dynamique résidant en mémoire RAM sous Unix/Linux. Il détaille l'ordonnancement par le noyau via un modèle à trois états (Élu, Prêt, Bloqué), la structure des attributs et du PCB (PID, PPID, filiation depuis systemd/init), ainsi que le contrôle inter-processus par les signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT).

1. Du Programme Statique au Processus Dynamique : Cycle de Vie et Attributs

Pour qu'un ordinateur exécute du code, le système d'exploitation transforme un programme stocké passivement sur disque en un processus actif en mémoire RAM. L'ordonnanceur du noyau gère cette exécution en attribuant le processeur aux processus selon un modèle à trois états fondamentaux : Élu (le processus s'exécute sur le CPU), Prêt (le processus attend qu'on lui attribue du temps CPU), et Bloqué (le processus attend la fin d'une opération d'E/S ou d'un événement externe). Chaque processus possède un identifiant unique (PID) et un identifiant de parent (PPID), formant une hiérarchie stricte dont la racine est le processus systemd ou init (PID 1).

- Programme vs Processus : Fichier passif sur disque vs instance active en RAM.
- Multitâche préemptif : Allocation dynamique de quanta de temps CPU par le noyau.
- Bloc de Contrôle du Processus (PCB) : Structure noyau contenant PID, PPID, état, registres et UID.
- Arborescence PID/PPID : Filiation hiérarchique issue de systemd/init (PID 1).

Cas concret enseignant : *L'analogie du livre de cuisine : Le programme sur disque dur est un livre de recettes rangé dans une bibliothèque (statique, passif, ne consommant ni énergie ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, consommant de l'espace sur le plan de travail/RAM et du temps de cuisson/CPU).*

2. Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux constituent la forme de communication inter-processus la plus élémentaire sous Unix. Lorsqu'un signal est émis, le noyau interrompt le flux d'exécution normal pour exécuter le gestionnaire associé. Bien que la plupart des signaux puissent être interceptés ou ignorés par l'application, la norme POSIX garantit l'autorité absolue de l'administrateur système via des signaux inïgnorables et incapturables comme SIGKILL et SIGSTOP.

- Signal POSIX : Interruption logicielle asynchrone transmise à un processus.
- Gestionnaire de signal (Signal Handler) : Fonction permettant d'intercepter (catch) ou ignorer un signal.
- SIGINT (Signal 2) : Interruption interactive déclenchée par Ctrl + C.
- SIGTERM (Signal 15) : Demande de terminaison propre par défaut.
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel géré par le noyau.
- Job Control : SIGSTOP (19) pour suspendre et SIGCONT (18) pour reprendre.

Cas concret enseignant : *L'analogie de l'expulsion de l'immeuble : SIGTERM (15) est le gardien qui frappe poliment pour vous demander de quitter les lieux (vous avez le temps de ranger vos affaires et sauvegarder). SIGKILL (9) est une intervention d'urgence qui défonce la porte et vous expulse instantanément sans sommation.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Parce que sans le tiret '-', le shell interprète le premier argument '9' comme le PID du processus à cibler et non comme le numéro du signal. Le 'kill' par défaut envoie le signal 15 (SIGTERM) en cherchant un processus de PID 9, ce qui échoue ou cible un mauvais processus.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus zombie est un processus terminé dont l'entrée dans la table des processus (le PCB) subsiste car le parent n'a pas encore lu son code de retour via l'appel système 'wait()'. Il est impossible de tuer un zombie avec 'kill -9' car il est déjà mort ; la solution consiste à éliminer ou relancer son processus parent pour que le noyau nettoie la table.

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash Linux, un étudiant saisit la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le traitement réalisé lors de l'envoi du signal -15 et décrivez le résultat de la vérification par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' en fin de ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background), libérant immédiatement l'invite de commande tout en affichant le PID. Étape 2 : La commande 'ps -ef | grep sleep' liste les processus actifs et filtre ceux correspondants à 'sleep' pour récupérer son PID exact. Étape 3 : La commande 'kill -15 ' envoie le signal SIGTERM au processus. L'utilitaire sleep s'arrête proprement en libérant ses ressources.

Résultat : Le processus sleep est définitivement arrêté. Une nouvelle exécution de 'ps -ef | grep sleep' confirme sa disparition totale de la table des processus.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : systemd ou init, avec le PID 1.

Q : Quel signal POSIX est transmis au processus s'exécutant au premier plan lorsqu'un utilisateur presse la combinaison 'Ctrl + C' ?

R : SIGINT (Signal 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté ni bloqué par un programme informatique ?

R : Parce qu'il est traité directement par le noyau pour garantir à l'administrateur un moyen infaillible de stopper un processus bloqué ou malveillant.